

Topic 2: Setting Up the C++ Development Environment

2.1 What is a Compiler?

- A compiler reads your source code (.cpp file) and translates it into machine code the CPU can run
- The translation happens all at once before the program runs — unlike an interpreter (Python) which runs line by line
- If the code has syntax errors, the compiler reports them with line numbers and produces no output
- If there are no errors, the compiler produces an executable file (.exe on Windows)
- The GNU C++ compiler (g++) is free and included with MinGW on Windows

2.2 What is an IDE?

- An IDE (Integrated Development Environment) combines everything a programmer needs in one place
- It includes: a code editor with syntax highlighting and auto-complete, a built-in compiler, a debugger, and a file manager
- Instead of using separate tools, you write, compile, and run code all from within the IDE
- Popular IDEs: Visual Studio, VS Code, Code::Blocks, Dev-C++, CLion

2.3 Installing Visual Studio (Recommended for Beginners)

- Download Visual Studio Community (completely free for students) from visualstudio.microsoft.com
- Run the installer and select the workload "Desktop development with C++"
- Click Install — this automatically sets up the MSVC compiler, debugger, and all required C++ tools
- No separate compiler download or PATH configuration needed — everything is set up automatically
- What is included: MSVC compiler, IntelliSense (auto-complete + error highlighting), built-in debugger, Solution Explorer

2.4 Creating and Running Your First C++ Project

- Open Visual Studio → click "Create a new project"

- Select "Console App" under C++ → click Next
- Give the project a name, choose a save location, click Create
- Visual Studio opens a ready-to-use main.cpp file with a basic Hello World program
- Press F5 to compile and run with the debugger — a console window shows the output
- Press Ctrl+F5 to run without the debugger (console stays open after the program finishes)

2.5 Structure of a C++ Program

- `#include <iostream>` : tells the preprocessor to include input/output functions
- `using namespace std;` : avoids writing `std::` before every standard library function
- `int main()` : every C++ program must have exactly one `main()` — execution starts here
- `{` and `}` : define the beginning and end of a code block — every opening brace must match a closing one
- Every statement ends with a semicolon `;` — missing semicolon is the most common beginner error
- `return 0;` : signals to the OS that the program ended successfully
- C++ is case-sensitive: `Main` is not `main`, `Cout` is not `cout`
- Indentation is ignored by the compiler but essential for readable code

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

2.6 Steps to Write and Run Any C++ Program

- Step 1: Write source code in a .cpp file
- Step 2: Compiler reads the file and checks for syntax errors
- Step 3: If no errors, compiler produces an object file (.o) with machine code
- Step 4: Linker combines the object file with library code to produce the final executable (.exe)
- Step 5: Run the executable — output appears in the terminal or console

2.7 Types of Errors

- Syntax errors: violations of C++ grammar rules — caught by the compiler before the program runs
 - Runtime errors: occur while the program is running — dividing by zero, accessing invalid memory
 - Logic errors: program compiles and runs but produces wrong results due to a reasoning mistake
-